



Hands-On Activity: More practice with SQL



Activity Overview

In previous lessons, you learned how to work with data in spreadsheets. Now, you'll practice using SQL to work with data in databases. By the time you complete this activity, you'll be able to use SQL to write queries that retrieve data from databases. Further, you'll practice navigating large public datasets in BigQuery in order to become proficient in SQL queries—an essential skill for your future career as a data analyst.

Review the following scenario. Then complete the step-by-step instructions.

In this activity, you'll query a public dataset that contains information about the trees in New York City in 2005. You'll also use the `avg` function to determine the average

diameter of all NYC trees in 2005. You'll then have the opportunity to write queries using the data for 1995 and 2015 to compare the average tree diameters.

Follow the instructions to complete each step of the activity. Then answer the questions at the end of the activity before going to the next course item.

1. Log in to [BigQuery Sandbox](#). On the BigQuery page, click the **Go to BigQuery** button.

If you have a [free-of-charge trial version of BigQuery](#), you can use that instead.

Note: BigQuery Sandbox frequently updates its user interface. The latest changes may not be reflected in the screenshots presented in this activity, but the principles remain the same. Adapting to changes in software updates is an essential skill for data analysts, and it's helpful for you to practice troubleshooting. You can also reach out to your community of learners on the discussion forum for help.

2. If you haven't done so already, **create a BigQuery project**. (If you have a project, select it in the **Explorer** pane.)

a. In the BigQuery console, select the dropdown list to the right of the Google Cloud logo to open the **Select a project** dialog box.



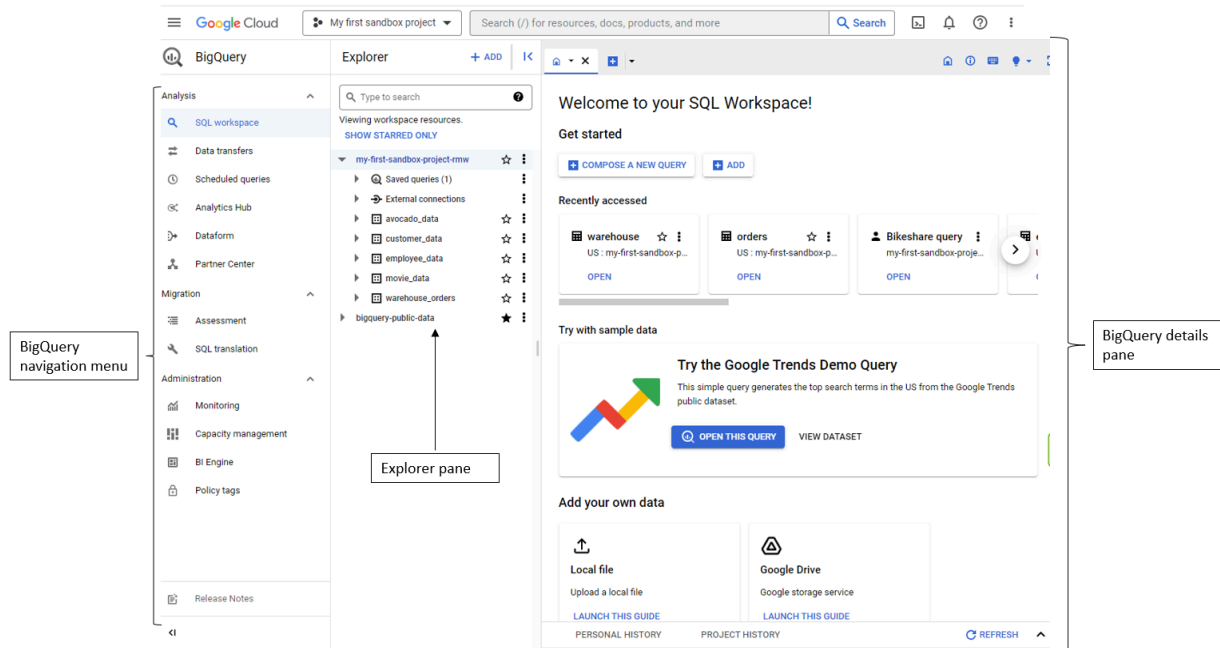
b. In the **Select a project** dialog box, select the **CREATE PROJECT** button.

c. Give your project a name that will help you identify it later. This can be a unique project ID or use an auto-generated one. You do not need to select an organization.

d. Select the **CREATE** button to create the project.

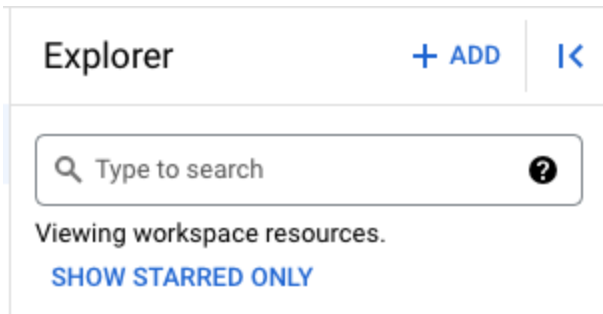
3. The three main sections of BigQuery are now onscreen: the **BigQuery navigation menu**; the **Explorer** pane, which you can use to search for public datasets and open projects; and the **Details** pane, which shows details of the database or dataset you've selected in the **Explorer** pane and displays windows for you to enter queries.

Notice that you can use the <| symbol in the BigQuery **navigation** menu section to collapse it. There is a similar symbol to collapse the **Explorer** pane.

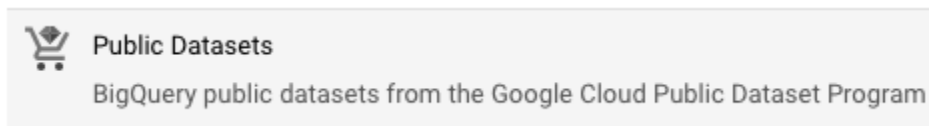


Follow these steps to find and select the NYC Trees dataset for this activity:

1. In the **Explorer** pane, select the **+ ADD** button.



2. In the **Add** box that pops up, scroll down the **Additional sources** list. Select **Public Datasets**.



3. A new box opens where you can search public datasets that are available through Google Cloud. In the **Search Marketplace** text box, search for **New York City Trees**.

Filter Type to filter

Category

Analytics (24)

Big data (18)

Databases (5)

Machine learning (4)

Maps (7)

Type

Datasets

Price

Free (208)

208 results



About COVID-19 Public Datasets

BigQuery Public Datasets Program

Getting started with COVID-19 Public Datasets

Cymbal

About Cymbal: Google Cloud's demo brand

Cymbal Group

Synthetic datasets across industries showcasing Google Cloud.



AFSC Open Data Portal NOAA

Fisheries research data for the Alaska region



American Community Survey (ACS)

United States Census Bureau

Detailed US demographic data at various geographic resolutions

broadstreet

Area Deprivation Index (ADI) BroadStreet

ADI: An index of socioeconomic status for communities



Austin Crime Data City of Austin

City of Austin crime data for 2014 and 2015



Band Protocol Data

Cloud Public Datasets - Finance

Band Protocol data loaded into BigQuery



Births Data Summary

Centers for Disease Control

Nativity Data from CDC Births



Bitcoin Cash Cryptocurrency Dataset

Bitcoin Cash

The Bitcoin Cash blockchain loaded to BigQuery

4. Select the search result **NYC Street Trees**, then select the **View Dataset** button.



NYC Street Trees

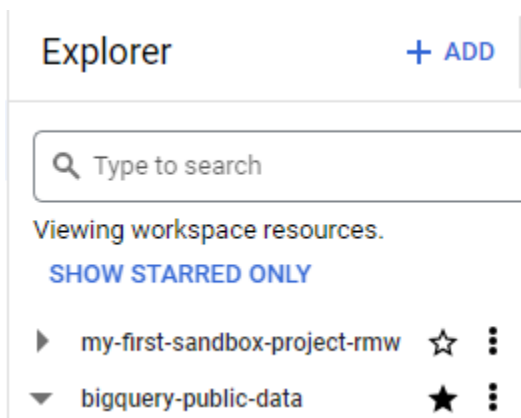
[City of New York](#)

New York City Street Tree Census data

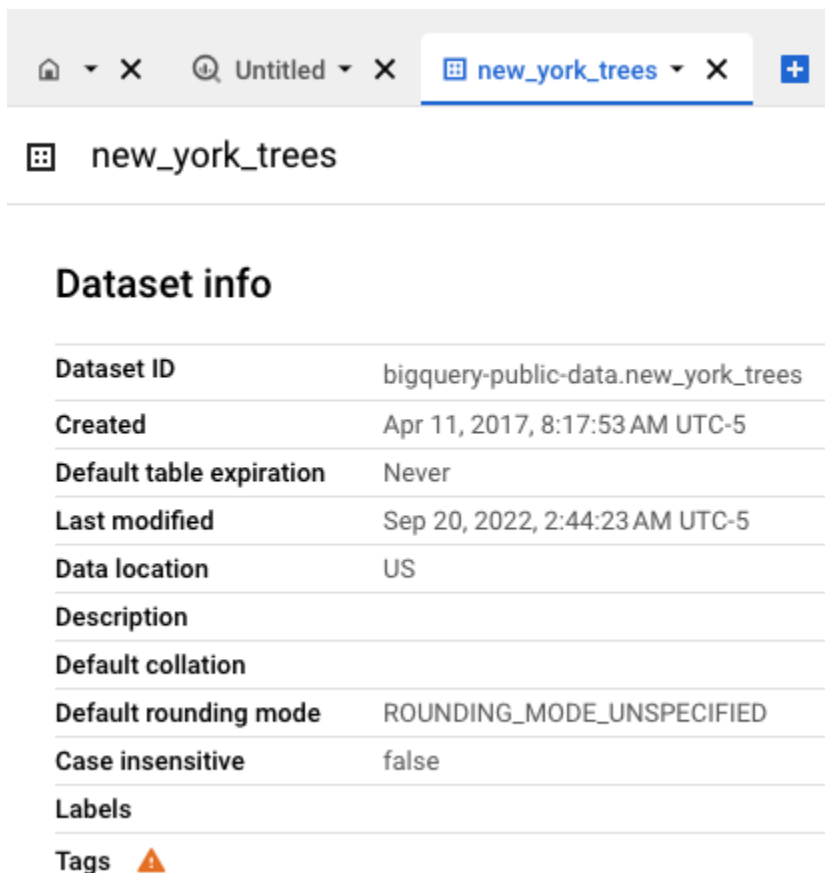
[VIEW DATASET](#)

Heading is NYC Street Trees. Link to City of New York provided. Subheading is New York City Street Tree Census data. Select View dataset button to link to dataset.

5. Google Cloud opens a new browser tab displaying BigQuery with the `bigquery-public-data` collection open in the **Explorer** pane. To ensure the `bigquery-public-data` database remains in your project's **Explorer** pane, select the star next to the dataset.



6. The BigQuery **Details** pane contains information about the `new_york_trees` dataset. This information includes the date the dataset was created, when it was last modified, and the Dataset ID.



The screenshot shows the BigQuery interface with the 'new_york_trees' dataset selected in the top navigation bar. Below the navigation bar, the 'new_york_trees' dataset is listed. The 'Dataset info' section is expanded, displaying a table of dataset properties.

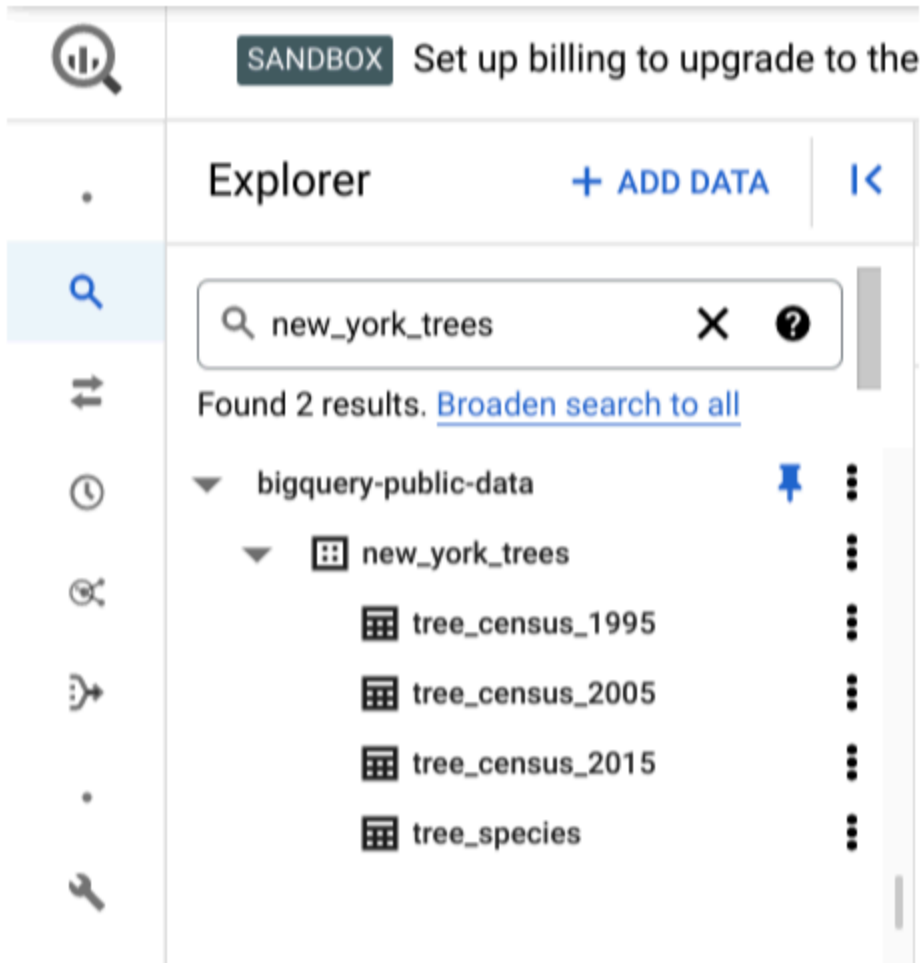
Dataset info	
Dataset ID	bigquery-public-data.new_york_trees
Created	Apr 11, 2017, 8:17:53 AM UTC-5
Default table expiration	Never
Last modified	Sep 20, 2022, 2:44:23 AM UTC-5
Data location	US
Description	
Default collation	
Default rounding mode	ROUNDING_MODE_UNSPECIFIED
Case insensitive	false
Labels	
Tags	

The Details pane displaying the `new_york_trees` data description including the dataset ID, when it was created, default table expiration, when it was last modified, the data location, description, the default collation, the default rounding mode, case insensitive, the labels, and the tags.

1. In the **Explorer** pane, select the arrow next to the `new_york_trees` dataset to display the tables it contains.

Note: If the `new_york_trees` dataset is not in the **Explorer** pane, type `new_york_trees` into the **Search** text box in the **Explorer** pane. (This will work if you have pinned `bigquery-public-data` in the **Explorer** pane.) If search doesn't return the needed results, follow the steps above to search for the `new_york_trees` dataset.

2. Notice that the `new_york_trees` dataset contains three tree census tables from 1995, 2005, and 2015. It also contains a table that lists the tree species.



In the Explorer pane, bigquery-public-data is open and the new_york_trees dataset is expanded to show the tables in the dataset are: tree_census_1995, tree_census_2005, tree_census_2015, and tree_species.

These are all tables contained in the dataset. Now, examine the data for all trees cataloged in New York City for three specific years.

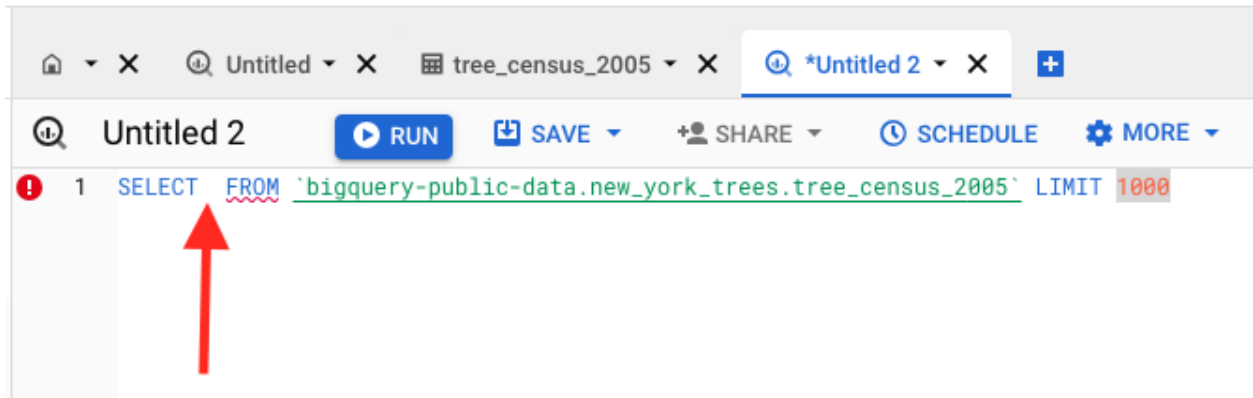
3. Select the `tree_census_2005` table. BigQuery displays the table's structure in the **Details** pane.

4. In the **Details** pane, select **Query > In new tab** to open a new query window.

The screenshot shows the Google Cloud BigQuery interface. On the left is the 'Explorer' pane with a search bar and a list of workspace resources. The 'tree_census_2005' table is selected. The main pane shows the 'Details' view for the 'tree_census_2005' table. A red box highlights the 'QUERY' dropdown menu, and a red arrow points to the 'In new tab' option. The table structure is displayed in a table format.

Field name	Type	Mode	Key	Collation	Default Value	Policy
☐ objectid	INTEGER	REQUIRED				
☐ cen_year	INTEGER	NULLABLE				
☐ tree_dbh	INTEGER	NULLABLE				
☐ tree_loc	STRING	NULLABLE				
☐ pit_type	STRING	NULLABLE				
☐ soil_lvl	STRING	NULLABLE				
☐ status	STRING	NULLABLE				
☐ spc_latin	STRING	NULLABLE				
☐ spc_common	STRING	NULLABLE				
☐ vert_other	BOOLEAN	NULLABLE				
☐ vert_pgrd	BOOLEAN	NULLABLE				

5. Notice that BigQuery populates the **Query Window** with a **SELECT** query. This query is incomplete because it doesn't contain anything in between **SELECT** and **FROM**.



This **SELECT** statement in the **Details** pane is incomplete because the columns to display have not been specified. So, either list the columns separated by commas or use the asterisk to have BigQuery return all columns in the table.

1. Type an asterisk * after the **SELECT** command in line one of the **Query Editor**. Your query should now read **SELECT * FROM** followed by your table location. This command tells BigQuery to return all of the columns in the **tree_census_2005** table.

2. In the **Query Editor**, select the **Run** button to run the query. The results will be displayed as a table in the **Query Results** pane below the **Query Editor**.

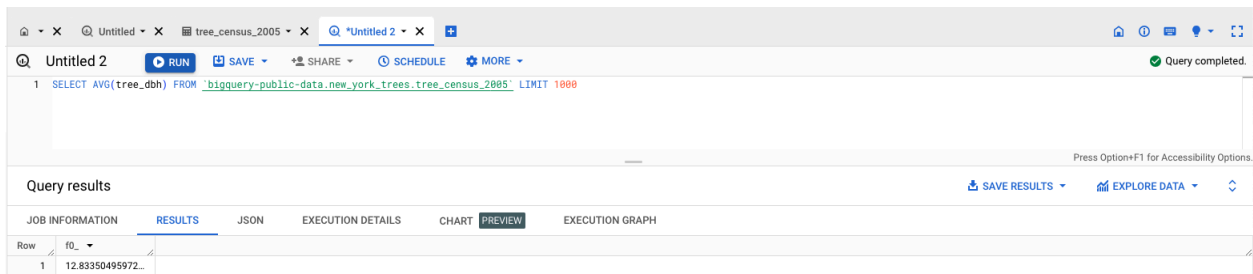
The screenshot shows the Google Cloud BigQuery Query Editor interface. At the top, there's a toolbar with buttons for 'RUN', 'SAVE', 'SHARE', 'SCHEDULE', and 'MORE'. Below the toolbar, the query editor contains a single query: `SELECT * FROM 'bigquery-public-data.new_york_trees.tree_census_2005' LIMIT 1000`. Below the query editor, the 'Query results' pane is active, displaying a table with 16 rows and 11 columns. The columns are: Row, objectid, cen_year, tree_dbh, tree_loc, pit_type, soil_lvl, status, spc_latin, and spc_common. The table shows data for various tree specimens, including their IDs, census years, diameters, locations, pit types, soil levels, and species names. The interface also includes tabs for 'JOB INFORMATION', 'RESULTS', 'JSON', 'EXECUTION DETAILS', 'CHART', 'PREVIEW', and 'EXECUTION GRAPH'. The 'PREVIEW' tab is currently selected. At the bottom right, there's a 'Results per page' dropdown set to 50, and a status bar indicating '1 - 50 of 1000'.

Row	objectid	cen_year	tree_dbh	tree_loc	pit_type	soil_lvl	status	spc_latin	spc_common
1	1091298	0	7	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
2	1091299	0	7	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
3	1091300	0	6	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
4	1091301	0	5	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
5	1092323	0	20	Side	Continuous Pit	Level	Good	GLEDITSIA TRIACANTHOS	HONEYLOCUST
6	1092327	0	10	Side	Continuous Pit	Level	Excellent	GLEDITSIA TRIACANTHOS	HONEYLOCUST
7	1092328	0	10	Side	Continuous Pit	Level	Excellent	GLEDITSIA TRIACANTHOS	HONEYLOCUST
8	1092330	0	6	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
9	1092331	0	5	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
10	1092332	0	5	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
11	1092333	0	7	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
12	1092334	0	7	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
13	1092335	0	7	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
14	1092336	0	6	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
15	1092337	0	6	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN
16	1092338	0	5	Front	Lawn	Level	Good	UNKNOWN	UNKNOWN

Results in the preview mode with columns with data populated including the row, object ID, cen_year, tree_dbh, tree_loc, pit_type, soil_lvl, status, spc_latin, and spc_common.

This query returns all columns for the first 1,000 rows from the table. BigQuery returns only the first 1,000 rows because the **SELECT** query includes a **LIMIT 1000** clause. This limits the rows returned to reduce the processing time required.

3. Next, write a query to find out the average diameter of all NYC trees in 2005. On line 1, replace the * after the **SELECT** command with **AVG(tree_dbh)**. Select the **Run** button to execute the query.



The screenshot shows the Google Cloud BigQuery interface. At the top, there's a query editor with the following SQL query: `1 SELECT AVG(tree_dbh) FROM `bigquery-public-data.new_york_trees.tree_census_2005` LIMIT 1000`. Below the editor, the 'Query results' section is active, displaying a table with one row and one column. The table has a header row with 'Row' and 'f0_'. The first row contains the value '1' and '12.83350495972...'. The interface also includes tabs for 'JOB INFORMATION', 'RESULTS', 'JSON', 'EXECUTION DETAILS', 'CHART', 'PREVIEW', and 'EXECUTION GRAPH'. A 'Query completed' status is visible in the top right corner.

Row	f0_
1	12.83350495972...

This returns your answer, **12.833** (which means the average diameter of NYC trees in 2005 was 12.833 inches).

Now, come up with some questions and answer them with your own SQL queries. For example, query the 1995 and the 2015 tables to find the average diameter of trees. You can then compare the average diameter of the trees in all three datasets to determine whether the trees in NYC have grown on average. Note that the field name for tree diameter in the **tree_census_1995** table is **diameter**.

You are also free to choose another publicly available dataset in BigQuery and write your own queries for extra practice—there are many interesting choices!

1.

Question 1

Reflection

You want to compare tree sizes in 2005 to tree sizes in 2015. In the activity above, you calculated the average diameter of NYC trees in 2005. Based on the `new_york_trees` dataset, what is the average diameter of NYC trees in 2015?

12.334

12.981

11.279

11.439

Correct

The average diameter of NYC trees in 2015 was 11.279 inches. The correct query to determine this information is:

```
SELECT avg(tree_dbh)
```

```
FROM `bigquery-public-data.new_york_trees.tree_census_2015`
```

1 / 1 point

2.

Question 2

During this activity, you practiced writing SQL queries to return information from datasets. In the text box below, write a 2-3 sentence (40-60 words) response to each of the following questions:

- What do you think might happen if you wrote each component of a query correctly, but rearranged the order?
- How can you use SQL queries to grow as a data analyst?

I think that if we wrote each component of the query correctly but rearranged the order, then it would give an error because, for instance, if we write `FROM tree_table SELECT *` instead of `SELECT * FROM tree_table`, then it will not run and will give an error.

As a data analyst, I use the `AVG` command to see the average from a particular column of a table, and there are many more queries that I use to find insight from data.

Feedback

Congratulations on completing this hands-on activity! A strong response would include how querying public datasets is a great way to practice SQL. Beyond that, consider the

following: Data analysts use SQL to interact with databases and view data they need to analyze. This is important knowledge that will prepare you for future courses and many aspects of your career as a data analyst. In upcoming activities, you will learn and practice writing more advanced queries, which will help you master SQL, an essential tool in every data analyst's toolkit.